ทักษะปัญญาประดิษฐ์กับการพัฒนาเว็บสมัยใหม่ แอปพลิเคชันมือถือ Flutter และ DevTools

Series of AI SKILL — Volume 04: AI for Modern Web, Flutter Mobile & High-Performance DevTools

ผู้ช่วยศาสตราจารย์ ดร.ชีวะ ทัศนา

สาขาวิชาฟิสิกส์ คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี

ศูนย์ความเป็นเลิศด้านสิ่งแวดล้อมและพลังงาน (LEQs) • พ.ศ. 2569

Contents

1 สถาปัตยกรรม Clean Architecture และการพัฒนาแอปพลิเคชัน Flutter **1**
1.1 การแบ่งเลเยอร์ตามหลักการออกแบบ Clean Architecture 1
1.2 การจัดการสถานะด้วย Provider และ ChangeNotifier 2

2 สถาปัตยกรรมเว็บสมัยใหม่และการประมวลผล 60 FPS **3**
2.1 เทคโนโลยี CSS ยุคใหม่ 3
2.2 การป้องกันปัญหา Garbage Collection Trashing . 4

3 การดีบั๊กขั้นสูงด้วย Chrome DevTools และการปรับแต่ง Core Web Vitals **5**
3.1 การตรวจจับการรั่วไหลของหน่วยความจำใน JavaScript 5
3.2 การปรับแต่งตัวชี้วัดความเร็ว Largest Contentful Paint 6

บทที่ 1

สถาปัตยกรรม Clean Architecture และการพัฒนาแอปพลิเคชัน Flutter

การพัฒนาแอปพลิเคชันโมบายล์ระดับวิชาชีพจำเป็นต้องมีโครงสร้างโค้ดที่สามารถขยายขนาดได้ง่าย (Scalability) บำรุงรักษาง่าย และแยกความรับผิดชอบของแต่ละส่วนออกจากกันอย่างชัดเจนตามหลัก Clean Architecture

1.1 การแบ่งเลเยอร์ตามหลักการออกแบบ Clean Architecture

โครงสร้างของระบบถูกแบ่งออกเป็น 3 เลเยอร์หลัก:

1. **Presentation Layer:** จัดการหน้าจอ UI, วิดเจ็ต, และ View-Model ที่เชื่อมต่อกับผู้ใช้
2. **Domain Layer:** แกนกลางของระบบที่บรรจุ Entity และ Use Case ทางธุรกิจ โดยไม่ขึ้นต่อเฟรมเวิร์กภายนอก
3. **Data Layer:** ติดต่อกับแหล่งข้อมูลภายนอก เช่น REST API, ฐานข้อมูลในเครื่อง SQLite หรือฮาร์ดแวร์บลูทูธ (BLE)

1.2 การจัดการสถานะด้วย Provider และ ChangeNotifier

การไหลของข้อมูลในแอปพลิเคชันใช้รูปแบบทางเดียว (Unidirectional Data Flow):

$$\text{User Action} \longrightarrow \text{ViewModel.execute()} \longrightarrow \text{State Mutation} \longrightarrow \text{UI Re-rend} \tag{1.1}$$

เมื่อสถานะภายในโมเดลเกิดการเปลี่ยนแปลง คำสั่ง `notifyListeners()` จะสั่งให้เฉพาะวิดเจ็ตที่มีการเรียกใช้ `Consumer` หรือ `context.watch()` เท่านั้นทำการเรนเดอร์ใหม่ ช่วยลดการสิ้นเปลืองทรัพยากร CPU ของสมาร์ทโฟน

บทที่ 2

สถาปัตยกรรมเว็บสมัยใหม่และการประมวลผล 60 FPS

เว็บเบราว์เซอร์ยุคใหม่มีขีดความสามารถเทียบเท่าเนทีฟแอปพลิเคชัน การนำเทคโนโลยี CSS ยุคใหม่ร่วมกับเอนจินกราฟิกส์ ช่วยให้สร้างอินเทอร์เฟซที่สวยงามและตอบสนองได้แบบทันทีทันใด

2.1 เทคโนโลยี CSS ยุคใหม่

การออกแบบเลย์เอาต์ที่ยืดหยุ่นในปัจจุบัน ไม่จำเป็นต้องพึ่งพา JavaScript ในการดักจับขนาดหน้าจอ:

- **Container Queries (`@container`):** ปรับเปลี่ยนสไตล์ตามขนาดของการ์ดหรือพาเนล ไม่จำกัดเฉพาะขนาดหน้าต่างเบราว์เซอร์
- **Parent Selector (`:has()`):** ควบคุมสไตล์ของพาเรนต์ตามสถานะขององค์ประกอบลูก
- **View Transitions API:** สร้างแอนิเมชันการสลับหน้าระหว่างการทดลองที่ลื่นไหลไร้รอยต่อ

2.2 การป้องกันปัญหา Garbage Collection Trashing

ในแอนิเมชันความเร็ว 60 เฟรมต่อวินาที แต่ละเฟรมมีเวลาประมวลผลเพียง 16.67 ms:

$$\Delta t_{\text{frame}} = \frac{1000\,\text{ms}}{60\,\text{FPS}} \approx 16.667\,\text{ms} \tag{2.1}$$

หากโค้ดสร้างออบเจกต์ชั่วคราวมากเกินไป ตัวเก็บขยะ (Garbage Collector) จะแทรกการทำงานและทำให้เวลาประมวลผลเฟรมเกิน 16.67 ms เกิดอาการภาพกระตุก (Jank) การใช้ ArrayBuffer ชนิด TypedArray และ Object Pool ช่วยขจัดปัญหานี้ได้อย่างสิ้นเชิง

บทที่ 3

การดีบั๊กขั้นสูงด้วย Chrome DevTools และการปรับแต่ง Core Web Vitals

การส่งมอบเว็บแอปพลิเคชันสู่สายตาผู้ใช้ จำเป็นต้องผ่านการตรวจสอบประสิทธิภาพเชิงลึกตามมาตรฐานของ Google Core Web Vitals และการตรวจสอบความถูกต้องของการเข้าถึง

3.1 การตรวจจับการรั่วไหลของหน่วยความจำใน JavaScript

การเกิดหน่วยความจำรั่วไหล (Memory Leak) มักเกิดจาก 3 สาเหตุหลัก:

1. **Detached DOM Nodes:** องค์ประกอบที่ถูกลบออกจากหน้าจอแต่ยังมีตัวแปรอ้างอิงอยู่

2. **Uncleared Event Listeners:** ตัวดักจับอีเวนต์ที่ไม่ได้ทำการถอดออกเมื่อปิดคอมโพเนนต์

3. **Retained Closures:** ฟังก์ชันปิดผนึกที่เก็บข้อมูลขนาดใหญ่ไว้ในขอบเขตการทำงาน

การใช้เครื่องมือ Heap Profiler ใน Chrome DevTools จะช่วยแสดงกราฟลำดับการอ้างอิง (Retainer Tree) ทำให้ชี้เป้าวัตถุต้นเหตุได้อย่างแม่นยำ

3.2 การปรับแต่งตัวชี้วัดความเร็ว Largest Contentful Paint

เกณฑ์ของ Core Web Vitals กำหนดว่าค่า LCP ต้องน้อยกว่า 2.5 วินาที:

$$\text{LCP} = T_{\text{TTFB}} + T_{\text{LoadDelay}} + T_{\text{LoadTime}} + T_{\text{RenderDelay}} < 2.5\,\text{s} \tag{3.1}$$

การเพิ่มคุณสมบัติ `fetchpriority="high"` ให้กับภาพฮีโร่และฟอนต์หลัก ร่วมกับการใช้เทคนิค Resource Preloading ช่วยลดค่า LCP ได้มากกว่าร้อยละ 40

บทที่ A

คลังรหัสและองค์ประกอบสกิลเว็บสมัยใหม่ Flutter และ DevTools

ภาคผนวกนี้รวบรวมข้อมูลจำเพาะ โค้ดต้นแบบ และสถาปัตยกรรมระดับองค์กรของสกิลการพัฒนาเว็บสมัยใหม่ แอปพลิเคชันมือถือ Flutter และเครื่องมือดีบั๊กประจำเล่มที่ 4

A.1 สกิล: flutter-app-architect

[แผนการสอน] ข้อมูลจำเพาะและเมทาดาทาของสกิล

ชื่อสกิล: `flutter-app-architect`
วัตถุประสงค์: วางโครงสร้างแอปพลิเคชัน Flutter แบบ Clean Architecture (MVVM/Provider) ปราศจากข้อผิดพลาด Static Linter
มาตรฐานการประเมิน: Multi-Screen Responsive, 0 Errors, 0 Warnings, Custom App Icon Pipeline

A.1.1 โค้ดแกนกลาง: แม่แบบ ViewModel และ State Management

```
class PhysicsExperimentViewModel extends ChangeNotifier {
  double _stringLength = 1.0; // meters
  double _gravity = 9.80665;  // m/s^2
  bool _isRunning = false;

  double get stringLength => _stringLength;
  double get period => 2 * 3.14159265 * Math.sqrt(
      _stringLength / _gravity);
  bool get isRunning => _isRunning;

  void updateLength(double newLength) {
    if (newLength > 0.1 && newLength <= 3.0) {
      _stringLength = newLength;
      notifyListeners(); // Triggers atomic UI re-render
    }
  }

  void toggleSimulation() {
    _isRunning = !_isRunning;
    notifyListeners();
  }
}
```

Listing A.1: Flutter ViewModel Architecture Blueprint

A.2 สกิล: modern-web-guidance

[แผนการสอน] ข้อมูลจำเพาะและเมทาดาทาของสกิล

ชื่อสกิล: `modern-web-guidance`
วัตถุประสงค์: กำหนดแนวปฏิบัติและมาตรฐาน API เว็บฟรอนต์เอนด์ยุคใหม่เพื่อประสิทธิภาพสูงสุด
ขอบเขตเทคโนโลยี: CSS Container Queries, View Transitions, Zero-GC RequestAnimationFrame

A.2.1 โค้ดแกนกลาง: สถาปัตยกรรม CSS Container Queries

```
/* Container Definition */
.lab-card-container {
```

```
  container-type: inline-size;
  container-name: labCard;
}

/* Dynamic Layout Adaptation */
@container labCard (min-width: 450px) {
  .lab-card {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 1.5rem;
    padding: 1.25rem;
  }
}
```

Listing A.2: Responsive Card Component Style

A.3 สกิล: memory-leak-debugging

[แผนการสอน] ข้อมูลจำเพาะและเมทาดาทาของสกิล

ชื่อสกิล: `memory-leak-debugging`
วัตถุประสงค์: วิเคราะห์ Heap Snapshot ตรวจจับหน่วยความจำรั่วไหลและระบุ Retainer Trees
ขอบเขตเทคโนโลยี: Chrome DevTools MCP, Heap Profiler, WeakMap Management

A.3.1 โค้ดแกนกลาง: การจัดเก็บตัวแปรอ้างอิงแบบอัตโนมัติด้วย WeakMap

```
// Prevents Detached DOM Node Leaks
const nodeMetadataCache = new WeakMap();

function registerElementData(element, metadata) {
  // If element is removed from DOM, GC reclaims metadata
      automatically
  nodeMetadataCache.set(element, metadata);
}
```

Listing A.3: Leak-Proof DOM Reference Cache